Domain-Driven Design

Deep Dive Learning Guide

Based on "Domain-Driven Design: Tackling Complexity in the Heart of Software"

by Eric Evans

Complete Guide with Go Examples

Table of Contents

1. Introduction to Domain-Driven Design

1.1 What is DDD?

1.2 When to Use DDD

1.3 Core Principles

2. Ubiquitous Language

2.1 Creating the Language

2.2 Language in Code

3. Tactical Design Patterns

3.1 Entities

3.2 Value Objects

3.3 Aggregates

3.4 Repositories

3.5 Factories

3.6 Domain Services

3.7 Domain Events

3.8 Specifications

4. Strategic Design

4.1 Bounded Contexts

4.2 Context Maps

4.3 Integration Patterns

5. Supple Design

5.1 Intention-Revealing Interfaces

5.2 Side-Effect-Free Functions

5.3 Assertions

6. Practical Example: E-Commerce System

7. Summary and Best Practices

1. Introduction to Domain-Driven Design

1.1 What is Domain-Driven Design?

Domain-Driven Design (DDD) is an approach to software development that centers the design on the core business domain. Rather than starting with technical considerations like database schemas or frameworks, DDD begins with understanding the business problem deeply and modeling it in code.

> **Definition: Domain-Driven Design**
>
> An approach to software development that focuses on understanding the business domain, creating a model that reflects domain knowledge, and implementing that model directly in code. The domain model becomes the heart of the software.

- The business domain is complex with intricate rules and workflows
- The project will evolve over time with changing requirements
- Domain experts are available to collaborate with developers
- The software's competitive advantage comes from business logic
- Multiple teams work on different parts of the system

> **When NOT to Use DDD**
>
> DDD may be overkill for simple CRUD applications, technical utilities, or systems where the domain is trivial. The overhead of creating a rich domain model may not be justified for simple data entry or reporting applications.

1.3 Core Principles

Table 1: Core Principles of Domain-Driven Design

Principle	Description
Focus on the Core Domain	Identify and invest in the most valuable part of the business
Collaborate with Domain Experts	Developers and experts create the model together
Ubiquitous Language	Use the same language in code, documentation, and conversation
Model-Driven Design	The code directly reflects the domain model
Continuous Refinement	The model evolves as understanding deepens

2. Ubiquitous Language

The Ubiquitous Language is perhaps the most important concept in DDD. It is a shared language used by all team members, including developers, domain experts, testers, and

business analysts. This language is based on the domain model and is used in all forms of communication.

> **Definition: Ubiquitous Language**
>
> A shared language based on the domain model, used by all team members to connect their activities with the software. The language is expressed in code, tests, documentation, and everyday conversation.

2.1 Creating the Language

Creating a Ubiquitous Language is an iterative process:

1. **Listen to domain experts** - Note the terms they use and how they describe processes
2. **Draw diagrams** - Visual representations help clarify concepts
3. **Question inconsistencies** - When experts use different terms, resolve the ambiguity
4. **Refine the model** - As understanding grows, refine both the language and the model
5. **Reflect in code** - Use the language terms as class names, method names, and variables

2.2 Language in Code

The Ubiquitous Language should be directly reflected in your code. Class names, method names, and variable names should all use the language of the domain.

Example: Poor vs Good Naming

Poor (Technical Language):

```
// Without Ubiquitous Language
type Order struct {
    ID        string
    CID       string  // What is CID?
    Amt       float64 // Amount of what?
    Dt        time.Time // Date of what?
    IsP       bool    // Is what processed?
}

func (o *Order) Process() error { ... }
```

Good (Domain Language):

```
// With Ubiquitous Language
type Order struct {
    OrderID       OrderID
    CustomerID    CustomerID
    TotalAmount   Money
    PlacedAt      time.Time
    Status        OrderStatus
}

func (o *Order) ConfirmPayment() error { ... }
func (o *Order) Ship() error { ... }
func (o *Order) Deliver() error { ... }
```

3. Tactical Design Patterns

Tactical design patterns are the building blocks used to construct the domain model. They help structure the code to reflect the domain concepts accurately.

3.1 Entities

> **Definition: Entity**
>
> An object that is defined by its identity rather than its attributes. Two entities with the same identity are considered the same object, even if their attributes differ. Entities have a lifecycle and can change over time.

Figure 2: *Comparison between Entity and Value Object*

Entities have three key characteristics:

- **Identity:** A unique identifier that persists throughout the object's lifecycle
- **Continuity:** The object maintains its identity even as attributes change
- **Mutability:** Attributes can change without changing the entity's identity

Go Implementation: Entity

```
package domain

import (
    "errors"
    "time"
)

// OrderID is a type-safe identifier for Orders
type OrderID string

// Order is an Entity - defined by its identity
type Order struct {
    orderID     OrderID
    customerID  CustomerID
    items       []OrderItem
    status      OrderStatus
    totalAmount Money
    placedAt    time.Time
    shippedAt   *time.Time
    deliveredAt *time.Time
}

// NewOrder creates a new Order entity
func  NewOrder(orderID  OrderID,  customerID  CustomerID,  items  []OrderItem)
(*Order, error) {
    if orderID == "" {
        return nil, errors.New("orderID cannot be empty")
    }
    if customerID == "" {
        return nil, errors.New("customerID cannot be empty")
    }
    if len(items) == 0 {
        return nil, errors.New("order must have at least one item")
    }

    order := &Order{
        orderID:    orderID,
        customerID: customerID,
        items:      items,
        status:     OrderStatusPending,
        placedAt:   time.Now(),
    }
    order.calculateTotal()
    return order, nil
}

// OrderID returns the entity's identity (never changes)
func (o *Order) OrderID() OrderID {
    return o.orderID
}

// Status returns the current status
```

```go
}

// ConfirmPayment changes the order status (entity can mutate)
func (o *Order) ConfirmPayment() error {
    if o.status != OrderStatusPending {
        return errors.New("only pending orders can be confirmed")
    }
    o.status = OrderStatusConfirmed
    return nil
}

// Ship marks the order as shipped
func (o *Order) Ship() error {
    if o.status != OrderStatusConfirmed {
        return errors.New("only confirmed orders can be shipped")
    }
    now := time.Now()
    o.shippedAt = &now
    o.status = OrderStatusShipped
    return nil
}

// Equals compares two orders by identity
func (o *Order) Equals(other *Order) bool {
    if other == nil {
        return false
    }
    return o.orderID == other.orderID
}

func (o *Order) calculateTotal() {
    var total Money
    for _, item := range o.items {
        total = total.Add(item.Subtotal())
    }
    o.totalAmount = total
}
```

3.2 Value Objects

> **Definition: Value Object**
>
> An object that is defined by its attributes rather than an identity. Two value objects with the same attributes are considered equal. Value objects should be immutable.

Value objects have three key characteristics:

- **No Identity:** Equality is based on attribute values, not identity
- **Immutability:** Once created, value objects cannot be changed
- **Replaceability:** To "change" a value object, you replace it with a new one

Go Implementation: Value Objects

```go
package domain

import (
    "errors"
    "fmt"
)

// Money is a Value Object - immutable, defined by its attributes
type Money struct {
    amount   int64  // Store in smallest unit (cents) to avoid floating point
    currency string
}

// NewMoney creates a new Money value object
func NewMoney(amount float64, currency string) (Money, error) {
    if currency == "" {
        return Money{}, errors.New("currency cannot be empty")
    }
    if amount < 0 {
        return Money{}, errors.New("amount cannot be negative")
    }
    return Money{
        amount:   int64(amount * 100),
        currency: currency,
    }, nil
}

// Add returns a new Money (immutable operation)
func (m Money) Add(other Money) (Money, error) {
    if m.currency != other.currency {
        return Money{}, errors.New("cannot add different currencies")
    }
    return Money{
        amount:   m.amount + other.amount,
        currency: m.currency,
    }, nil
}

// Multiply returns a new Money with multiplied amount
func (m Money) Multiply(factor float64) Money {
    return Money{
        amount:   int64(float64(m.amount) * factor),
        currency: m.currency,
    }
}

// Equals compares two Money value objects
func (m Money) Equals(other Money) bool {
    return m.amount == other.amount && m.currency == other.currency
}

// String representation
```

```go
}

// Email is another Value Object
type Email string

func NewEmail(value string) (Email, error) {
    if value == "" {
        return "", errors.New("email cannot be empty")
    }
    // Add email validation
    return Email(value), nil
}

// Address is a composite Value Object
type Address struct {
    street  string
    city    string
    zipCode string
    country string
}

func NewAddress(street, city, zipCode, country string) (Address, error) {
    if street == "" || city == "" || zipCode == "" {
        return Address{}, errors.New("address fields cannot be empty")
    }
    return Address{
        street:  street,
        city:    city,
        zipCode: zipCode,
        country: country,
    }, nil
}

// Equals compares two Address value objects
func (a Address) Equals(other Address) bool {
    return a.street == other.street &&
        a.city == other.city &&
        a.zipCode == other.zipCode &&
        a.country == other.country
}
```

Value Object Benefits

- **Immutability ensures safety:** No unexpected changes
- **Easy to test:** Equality is straightforward
- **Can be shared:** Multiple entities can reference the same value object

- **Self-validating:** Invalid states are impossible to create

3.3 Aggregates

Definition: Aggregate

A cluster of associated objects that are treated as a single unit for data changes. Each aggregate has a root entity (Aggregate Root) and a boundary. External objects can only reference the aggregate root.

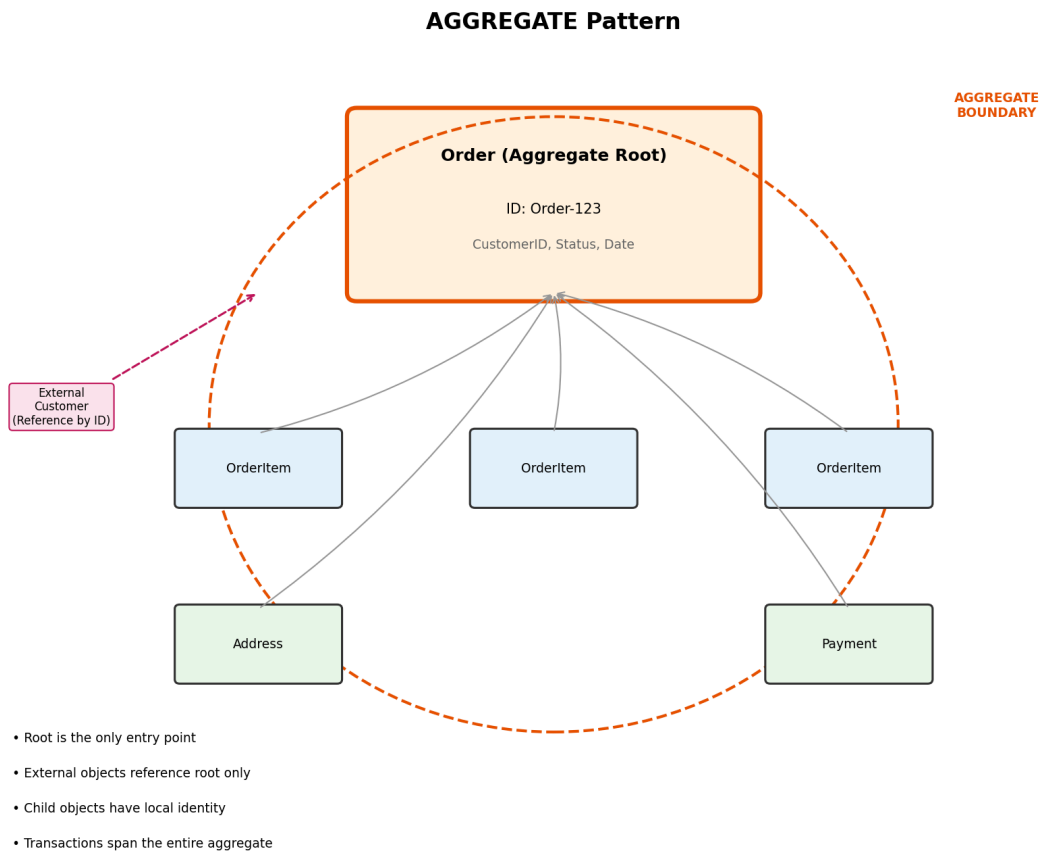

Figure 3: *Aggregate pattern with root and boundary*

Aggregate design rules:

- The aggregate root is the only entry point to the aggregate

Go Implementation: Aggregate

```
package domain

import (
    "errors"
    "time"
)

// Order is the Aggregate Root
type Order struct {
    orderID       OrderID
    customerID    CustomerID
    items         []OrderItem      // Child entities within aggregate
    shippingAddress Address        // Value object within aggregate
    status        OrderStatus
    createdAt     time.Time
}

// OrderItem is a child entity (local identity only)
type OrderItem struct {
    productID ProductID
    quantity  int
    unitPrice Money
}

// NewOrder creates a new Order aggregate
func NewOrder(orderID OrderID, customerID CustomerID,
              shippingAddress Address) (*Order, error) {
    if orderID == "" || customerID == "" {
        return nil, errors.New("orderID and customerID are required")
    }

    return &Order{
        orderID:         orderID,
        customerID:      customerID,
        items:           make([]OrderItem, 0),
        shippingAddress: shippingAddress,
        status:          OrderStatusPending,
        createdAt:       time.Now(),
    }, nil
}

// AddItem adds an item to the order (only through aggregate root)
func (o *Order) AddItem(productID ProductID, quantity int,
                        unitPrice Money) error {
    if o.status != OrderStatusPending {
        return errors.New("cannot modify confirmed order")
    }
    if quantity <= 0 {
        return errors.New("quantity must be positive")
    }

    // Check if product already exists
```

```
            // Update existing item
            o.items[i].quantity += quantity
            return nil
        }
    }

    // Add new item
    o.items = append(o.items, OrderItem{
        productID: productID,
        quantity:  quantity,
        unitPrice: unitPrice,
    })
    return nil
}

// RemoveItem removes an item from the order
func (o *Order) RemoveItem(productID ProductID) error {
    if o.status != OrderStatusPending {
        return errors.New("cannot modify confirmed order")
    }

    for i, item := range o.items {
        if item.productID == productID {
            o.items = append(o.items[:i], o.items[i+1:]...)
            return nil
        }
    }
    return errors.New("item not found")
}

// TotalAmount calculates the order total
func (o *Order) TotalAmount() Money {
    var total Money
    for _, item := range o.items {
        subtotal, _ := item.unitPrice.Multiply(float64(item.quantity))
        total, _ = total.Add(subtotal)
    }
    return total
}

// Confirm transitions the order to confirmed status
func (o *Order) Confirm() error {
    if len(o.items) == 0 {
        return errors.New("cannot confirm empty order")
    }
    if o.status != OrderStatusPending {
        return errors.New("only pending orders can be confirmed")
    }
    o.status = OrderStatusConfirmed
    return nil
}

// OrderID returns the aggregate root's identity
func (o *Order) OrderID() OrderID {
```

3.4 Repositories

> **Definition: Repository**
>
> A mechanism for encapsulating storage, retrieval, and search behavior which emulates a collection of objects. Repositories provide an interface that focuses on the domain model while hiding the details of data access.

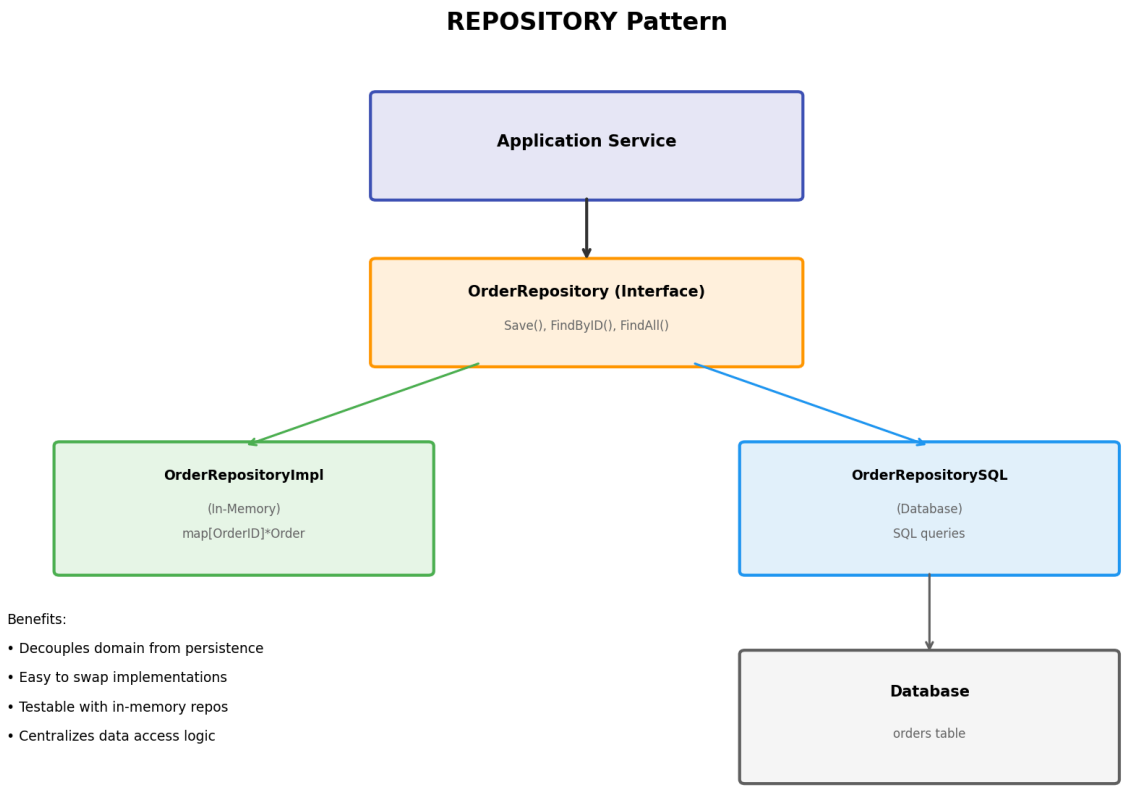

Figure 4: *Repository pattern decouples domain from persistence*

Go Implementation: Repository

```
package domain

import "context"

// OrderRepository is the interface - domain layer
type OrderRepository interface {
    Save(ctx context.Context, order *Order) error
    FindByID(ctx context.Context, id OrderID) (*Order, error)
      FindByCustomerID(ctx context.Context, customerID CustomerID) ([]*Order,
error)
    FindPending(ctx context.Context) ([]*Order, error)
    Delete(ctx context.Context, id OrderID) error
}

// In infrastructure layer - in-memory implementation
package infrastructure

type InMemoryOrderRepository struct {
    orders map[OrderID]*Order
}

func NewInMemoryOrderRepository() *InMemoryOrderRepository {
    return &InMemoryOrderRepository{
        orders: make(map[OrderID]*Order),
    }
}

func (r *InMemoryOrderRepository) Save(ctx context.Context, order *Order)
error {
    r.orders[order.OrderID()] = order
    return nil
}

func (r *InMemoryOrderRepository) FindByID(ctx context.Context,
    id OrderID) (*Order, error) {
    order, exists := r.orders[id]
    if !exists {
        return nil, ErrOrderNotFound
    }
    return order, nil
}

func (r *InMemoryOrderRepository) FindByCustomerID(ctx context.Context,
    customerID CustomerID) ([]*Order, error) {
    var result []*Order
    for _, order := range r.orders {
        if order.CustomerID() == customerID {
            result = append(result, order)
        }
    }
    return result, nil
}
```

```
error) {
    var result []*Order
    for _, order := range r.orders {
        if order.Status() == OrderStatusPending {
            result = append(result, order)
        }
    }
    return result, nil
}

func (r *InMemoryOrderRepository) Delete(ctx context.Context,
    id OrderID) error {
    delete(r.orders, id)
    return nil
}

// SQL implementation
package infrastructure

type SQLOrderRepository struct {
    db *sql.DB
}

func NewSQLOrderRepository(db *sql.DB) *SQLOrderRepository {
    return &SQLOrderRepository{db: db}
}

func (r *SQLOrderRepository) Save(ctx context.Context, order *Order) error {
    // SQL implementation
    query := `
        INSERT INTO orders (order_id, customer_id, status, created_at)
        VALUES ($1, $2, $3, $4)
        ON CONFLICT (order_id) DO UPDATE SET
            customer_id = EXCLUDED.customer_id,
            status = EXCLUDED.status
    `
    _, err := r.db.ExecContext(ctx, query,
        order.OrderID(),
        order.CustomerID(),
        order.Status(),
        order.CreatedAt(),
    )
    return err
}

func (r *SQLOrderRepository) FindByID(ctx context.Context,
    id OrderID) (*Order, error) {
    query := `SELECT order_id, customer_id, status, created_at
              FROM orders WHERE order_id = $1`
    row := r.db.QueryRowContext(ctx, query, id)

    var order Order
    err := row.Scan(&order.orderID, &order.customerID,
                    &order.status, &order.createdAt)
```

```
    }
    if err != nil {
        return nil, err
    }
    return &order, nil
}
```

3.5 Factories

Definition: Factory

A mechanism for creating complex objects and aggregates while encapsulating the creation logic. Factories ensure that created objects are in a valid state and maintain invariants.

Go Implementation: Factory

```
package domain

import (
    "context"
    "errors"
    "time"
)

// OrderFactory creates Order aggregates
type OrderFactory struct {
    idGenerator IDGenerator
}

func NewOrderFactory(idGenerator IDGenerator) *OrderFactory {
    return &OrderFactory{idGenerator: idGenerator}
}

// CreateOrder creates a new Order with all required setup
func (f *OrderFactory) CreateOrder(
    ctx context.Context,
    customerID CustomerID,
    shippingAddress Address,
    items []OrderItemInput,
) (*Order, error) {
    // Validate inputs
    if customerID == "" {
        return nil, errors.New("customer ID is required")
    }
    if len(items) == 0 {
        return nil, errors.New("order must have at least one item")
    }

    // Generate ID
    orderID, err := f.idGenerator.GenerateOrderID(ctx)
    if err != nil {
        return nil, err
    }

    // Create the order
    order, err := NewOrder(orderID, customerID, shippingAddress)
    if err != nil {
        return nil, err
    }

    // Add items
    for _, item := range items {
                    if  err  :=  order.AddItem(item.ProductID,  item.Quantity,
item.UnitPrice); err != nil {
            return nil, err
        }
    }
```

```
// OrderItemInput is a DTO for factory input
type OrderItemInput struct {
    ProductID ProductID
    Quantity  int
    UnitPrice Money
}

// IDGenerator interface for ID generation
type IDGenerator interface {
    GenerateOrderID(ctx context.Context) (OrderID, error)
    GenerateCustomerID(ctx context.Context) (CustomerID, error)
    GenerateProductID(ctx context.Context) (ProductID, error)
}

// UUIDGenerator implementation
type UUIDGenerator struct{}

func (g *UUIDGenerator) GenerateOrderID(ctx context.Context) (OrderID, error)
{
    return OrderID(uuid.New().String()), nil
}
```

3.6 Domain Services

Definition: Domain Service

A service that performs a significant business process that does not naturally belong to any entity or value object. Domain services contain business logic that involves multiple aggregates or concepts.

Go Implementation: Domain Service

```go
package domain

import (
    "context"
    "errors"
)

// PricingService calculates prices for orders
type PricingService struct {
    discountPolicy DiscountPolicy
}

func NewPricingService(discountPolicy DiscountPolicy) *PricingService {
    return &PricingService{discountPolicy: discountPolicy}
}

// CalculateOrderTotal calculates the total with discounts
func (s *PricingService) CalculateOrderTotal(
    ctx context.Context,
    items []OrderItem,
    customer Customer,
) (Money, error) {
    var subtotal Money

    // Calculate item subtotals
    for _, item := range items {
        itemTotal, err := item.unitPrice.Multiply(float64(item.quantity))
        if err != nil {
            return Money{}, err
        }
        subtotal, err = subtotal.Add(itemTotal)
        if err != nil {
            return Money{}, err
        }
    }

    // Apply customer discount
    discount := s.discountPolicy.CalculateDiscount(customer, subtotal)

    // Calculate final total
    total, err := subtotal.Subtract(discount)
    if err != nil {
        return Money{}, err
    }

    return total, nil
}

// TransferService handles money transfers between accounts
type TransferService struct {
    accountRepo AccountRepository
}
```

```
    return &TransferService{accountRepo: accountRepo}
}

// Transfer performs a transfer between two accounts
func (s *TransferService) Transfer(
    ctx context.Context,
    fromAccountID AccountID,
    toAccountID AccountID,
    amount Money,
) (*Transfer, error) {
    // Load both accounts
    fromAccount, err := s.accountRepo.FindByID(ctx, fromAccountID)
    if err != nil {
        return nil, err
    }

    toAccount, err := s.accountRepo.FindByID(ctx, toAccountID)
    if err != nil {
        return nil, err
    }

    // Validate transfer
    if !fromAccount.CanWithdraw(amount) {
        return nil, errors.New("insufficient funds")
    }

    // Perform transfer
    if err := fromAccount.Withdraw(amount); err != nil {
        return nil, err
    }

    if err := toAccount.Deposit(amount); err != nil {
        return nil, err
    }

    // Save changes
    if err := s.accountRepo.Save(ctx, fromAccount); err != nil {
        return nil, err
    }

    if err := s.accountRepo.Save(ctx, toAccount); err != nil {
        return nil, err
    }

    // Return transfer record
    return NewTransfer(fromAccountID, toAccountID, amount), nil
}
```

3.7 Domain Events

> **Definition: Domain Event**
>
> A domain event is something that happened in the domain that other parts of the system might be interested in. Events represent facts that have occurred and are immutable.

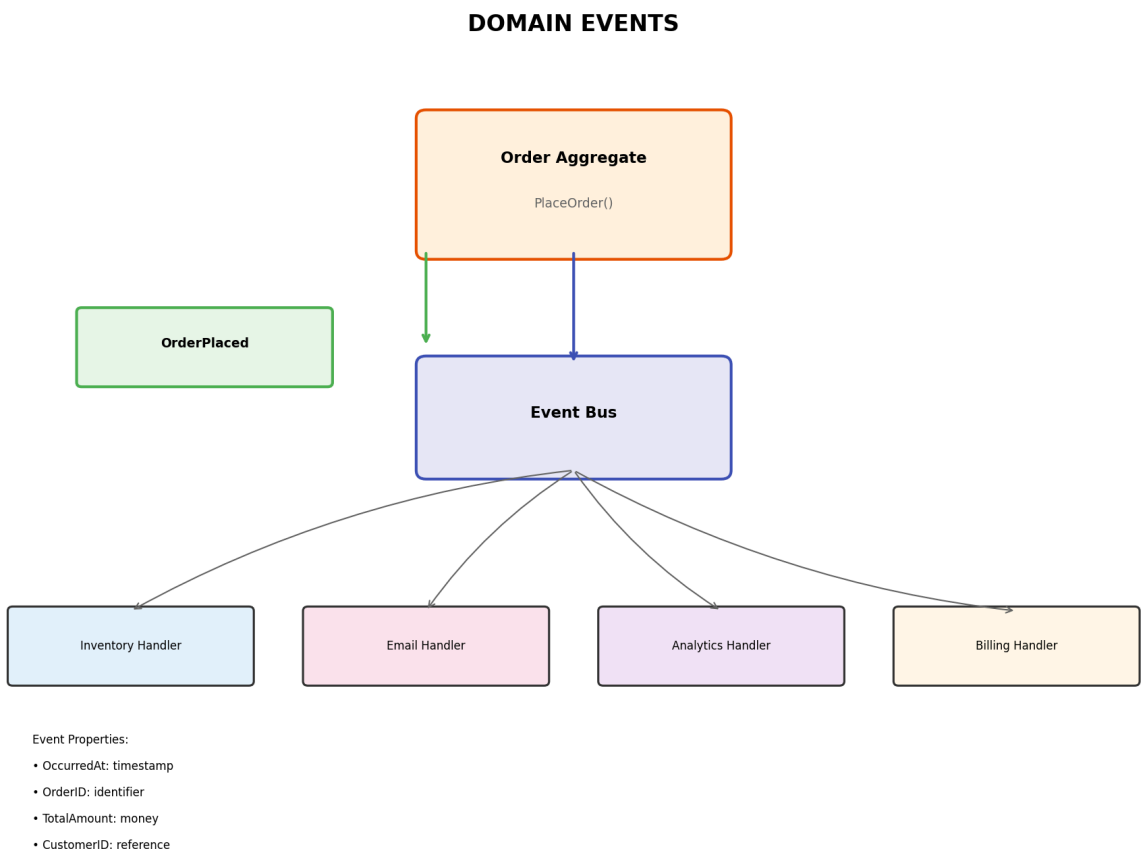

Figure 5: *Domain events enable loose coupling between aggregates*

Go Implementation: Domain Events

```
package domain

import (
    "context"
    "time"
)

// DomainEvent is the base interface for all domain events
type DomainEvent interface {
    EventName() string
    OccurredAt() time.Time
}

// OrderPlaced event
type OrderPlaced struct {
    occurredAt time.Time
    OrderID    OrderID
    CustomerID CustomerID
    TotalAmount Money
    ItemCount   int
}

func NewOrderPlaced(orderID OrderID, customerID CustomerID,
                    total Money, itemCount int) *OrderPlaced {
    return &OrderPlaced{
        occurredAt:  time.Now(),
        OrderID:     orderID,
        CustomerID:  customerID,
        TotalAmount: total,
        ItemCount:   itemCount,
    }
}

func (e *OrderPlaced) EventName() string {
    return "OrderPlaced"
}

func (e *OrderPlaced) OccurredAt() time.Time {
    return e.occurredAt
}

// OrderShipped event
type OrderShipped struct {
    occurredAt time.Time
    OrderID    OrderID
    ShippedAt  time.Time
    TrackingNumber string
}

func NewOrderShipped(orderID OrderID, trackingNumber string) *OrderShipped {
    now := time.Now()
    return &OrderShipped{
```

```
        ShippedAt:      now,
        TrackingNumber: trackingNumber,
    }
}

func (e *OrderShipped) EventName() string {
    return "OrderShipped"
}

func (e *OrderShipped) OccurredAt() time.Time {
    return e.occurredAt
}

// EventPublisher interface
type EventPublisher interface {
    Publish(ctx context.Context, event DomainEvent) error
    Subscribe(eventName string, handler EventHandler) error
}

type EventHandler func(ctx context.Context, event DomainEvent) error

// In-memory event bus implementation
type InMemoryEventBus struct {
    handlers map[string][]EventHandler
}

func NewInMemoryEventBus() *InMemoryEventBus {
    return &InMemoryEventBus{
        handlers: make(map[string][]EventHandler),
    }
}

func (b *InMemoryEventBus) Subscribe(eventName string, handler EventHandler)
error {
    b.handlers[eventName] = append(b.handlers[eventName], handler)
    return nil
}

func (b *InMemoryEventBus) Publish(ctx context.Context, event DomainEvent)
error {
    handlers := b.handlers[event.EventName()]
    for _, handler := range handlers {
        if err := handler(ctx, event); err != nil {
            // Log error but continue with other handlers
            continue
        }
    }
    return nil
}

// Order with event publishing
type Order struct {
    // ... other fields
    events []DomainEvent
```

```go
func (o *Order) Confirm() error {
    // ... validation
    o.status = OrderStatusConfirmed

    // Record domain event
    o.events = append(o.events, NewOrderPlaced(
        o.orderID,
        o.customerID,
        o.TotalAmount(),
        len(o.items),
    ))
    return nil
}

func (o *Order) PullEvents() []DomainEvent {
    events := o.events
    o.events = nil
    return events
}
```

3.8 Specifications

Definition: Specification

A specification is a predicate that determines whether an object satisfies certain criteria. Specifications encapsulate business rules in a reusable, composable way.

Go Implementation: Specifications

```
package domain

// Specification interface
type Specification[T any] interface {
    IsSatisfiedBy(candidate T) bool
    And(other Specification[T]) Specification[T]
    Or(other Specification[T]) Specification[T]
    Not() Specification[T]
}

// BaseSpecification provides default implementations
type BaseSpecification[T any] struct {
    predicate func(T) bool
}

func (s *BaseSpecification[T]) IsSatisfiedBy(candidate T) bool {
    return s.predicate(candidate)
}

func (s *BaseSpecification[T]) And(other Specification[T]) Specification[T] {
    return &BaseSpecification[T]{
        predicate: func(t T) bool {
            return s.IsSatisfiedBy(t) && other.IsSatisfiedBy(t)
        },
    }
}

func (s *BaseSpecification[T]) Or(other Specification[T]) Specification[T] {
    return &BaseSpecification[T]{
        predicate: func(t T) bool {
            return s.IsSatisfiedBy(t) || other.IsSatisfiedBy(t)
        },
    }
}

func (s *BaseSpecification[T]) Not() Specification[T] {
    return &BaseSpecification[T]{
        predicate: func(t T) bool {
            return !s.IsSatisfiedBy(t)
        },
    }
}

// Order specifications
type OrderSpecification Specification[*Order]

func NewHighValueOrderSpecification(minAmount Money) OrderSpecification {
    return &BaseSpecification[*Order]{
        predicate: func(o *Order) bool {
            return o.TotalAmount().amount >= minAmount.amount
        },
    }
```

```go
func NewPendingOrderSpecification() OrderSpecification {
    return &BaseSpecification[*Order]{
        predicate: func(o *Order) bool {
            return o.Status() == OrderStatusPending
        },
    }
}

func NewRecentOrderSpecification(since time.Time) OrderSpecification {
    return &BaseSpecification[*Order]{
        predicate: func(o *Order) bool {
            return o.CreatedAt().After(since)
        },
    }
}

// Usage example
func FindOrdersForReminder(orders []*Order) []*Order {
    // Orders that are pending AND old
    spec := NewPendingOrderSpecification().And(
        NewRecentOrderSpecification(time.Now().AddDate(0, 0, -7)).Not(),
    )

    var result []*Order
    for _, order := range orders {
        if spec.IsSatisfiedBy(order) {
            result = append(result, order)
        }
    }
    return result
}
```

4. Strategic Design

Strategic design deals with the big picture of how different parts of a system relate to each other. It provides patterns for dividing large systems into manageable parts and integrating them.

4.1 Bounded Contexts

> **Definition: Bounded Context**

> A bounded context is a specific responsibility with explicit boundaries that separate it from other contexts. Within a bounded context, a single domain model applies. Different contexts may have different models for similar concepts.

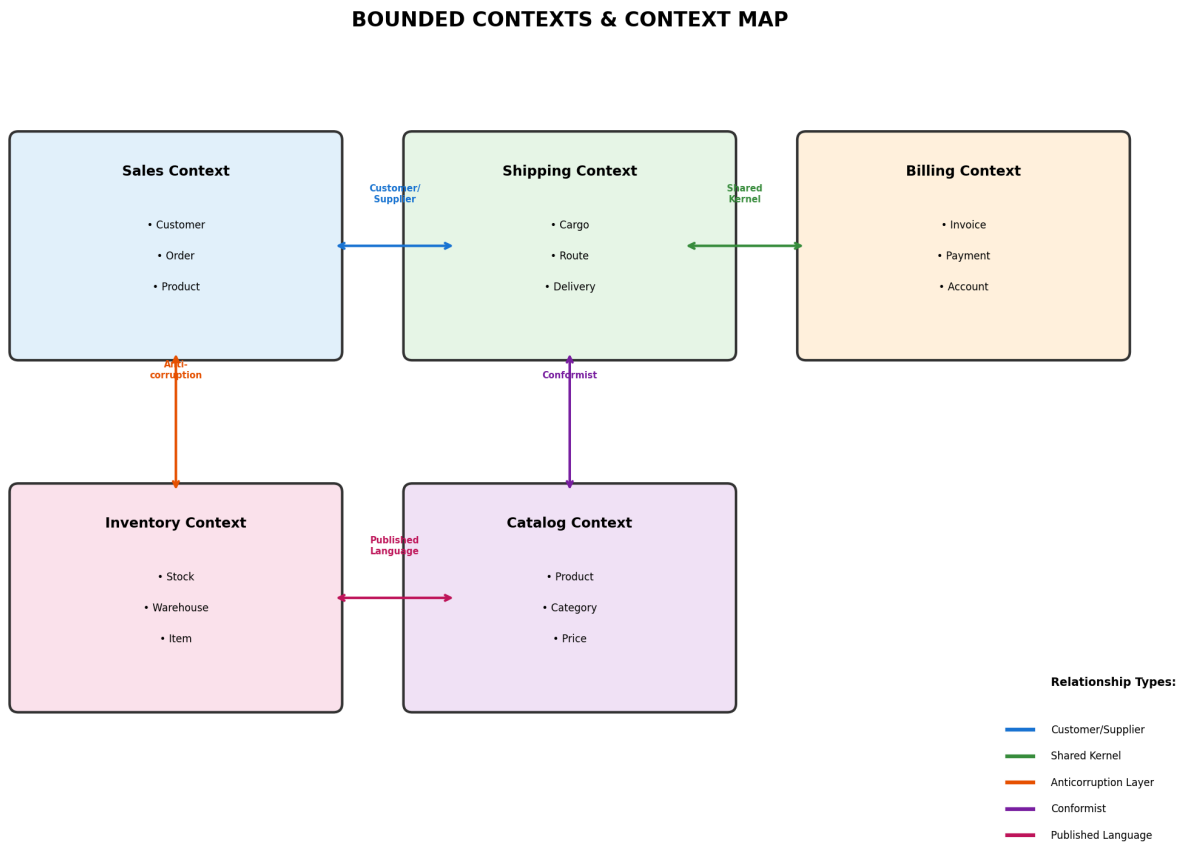

Figure 6: *Multiple bounded contexts with their own models*

Key aspects of bounded contexts:

- **Model Boundary:** Each context has its own domain model
- **Team Boundary:** Often aligns with team responsibilities
- **Code Boundary:** Clear separation in codebase
- **Database Boundary:** Can have separate databases

Example: E-Commerce Bounded Contexts

```
// Sales Context - Customer is a buyer
package sales

type Customer struct {
    ID        CustomerID
    Name      string
    Email     Email
    Addresses []Address
    Preferences CustomerPreferences
}

// Billing Context - Customer is a payer
package billing

type Customer struct {
    ID            CustomerID
    BillingAddress Address
    PaymentMethods []PaymentMethod
    CreditLimit   Money
    OutstandingBalance Money
}

// Shipping Context - Customer is a recipient
package shipping

type Customer struct {
    ID             CustomerID
    DeliveryAddress Address
    DeliveryInstructions string
    PreferredCarrier Carrier
}
```

4.2 Context Maps

A context map is a visual representation of the relationships between bounded contexts. It shows how contexts depend on each other and the nature of their relationships.

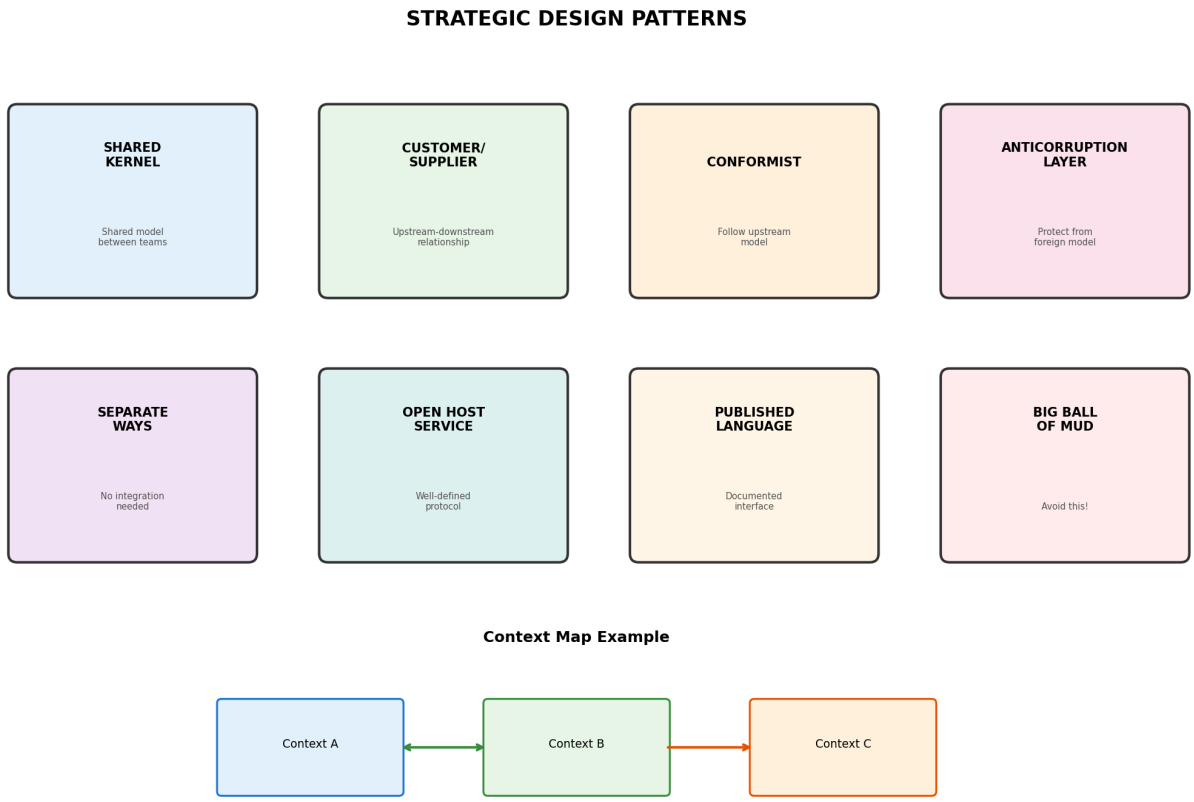

Figure 7: *Strategic design patterns for context relationships*

4.3 Integration Patterns

Table 2: Context Integration Patterns

Pattern	Use When	Description
Shared Kernel	Two teams share a small common model	Shared subset of domain model with coordinated changes
Customer/Supplier	Upstream and downstream relationship	Upstream prioritizes downstream needs
Conformist	Downstream must follow upstream	Downstream adopts upstream model completely

Published Language	Need documented interchange format	Explicit shared language for communication

Go Implementation: Anticorruption Layer

```go
package anticorruption

import (
    "legacy"
    "ourdomain"
)

// LegacyOrderTranslator translates between legacy and our domain
type LegacyOrderTranslator struct{}

func (t *LegacyOrderTranslator) ToOurDomain(
    legacyOrder legacy.Order,
) (*ourdomain.Order, error) {
    // Translate legacy format to our domain model
    orderID := ourdomain.OrderID(legacyOrder.OrderNumber)
    customerID := ourdomain.CustomerID(legacyOrder.CustomerCode)

    // Legacy uses different status values
    status := t.translateStatus(legacyOrder.Status)

    // Legacy stores amount as float, we use Money
    totalAmount, err := ourdomain.NewMoney(legacyOrder.Total, "USD")
    if err != nil {
        return nil, err
    }

    return ourdomain.NewOrder(orderID, customerID, totalAmount, status)
}

func (t *LegacyOrderTranslator) ToLegacyFormat(
    order *ourdomain.Order,
) legacy.Order {
    return legacy.Order{
        OrderNumber: string(order.OrderID()),
        CustomerCode: string(order.CustomerID()),
        Status: t.translateStatusToLegacy(order.Status()),
        Total:  order.TotalAmount().AsFloat(),
    }
}

func (t *LegacyOrderTranslator) translateStatus(
    legacyStatus string,
) ourdomain.OrderStatus {
    switch legacyStatus {
    case "P":
        return ourdomain.OrderStatusPending
    case "C":
        return ourdomain.OrderStatusConfirmed
    case "S":
        return ourdomain.OrderStatusShipped
    default:
        return ourdomain.OrderStatusUnknown
```

5. Supple Design

Supple design is about making the code easy to understand, change, and extend. These patterns help create designs that clearly communicate their intent.

5.1 Intention-Revealing Interfaces

Definition: Intention-Revealing Interface

Names of classes, methods, and variables should reveal their purpose and how they should be used. The interface should tell a story about what it does, not how it does it.

Example: Intention-Revealing vs Obscure

```
// Obscure - what does this do?
func (c *Calculator) Process(d Data) Result {
    return c.compute(d.a, d.b, d.c)
}

// Intention-revealing - clear what it does
func (o *Order) CalculateTotalWithDiscount(
    customer Customer,
    coupon Coupon,
) (Money, error) {
    subtotal := o.calculateItemSubtotals()
    customerDiscount := o.applyCustomerDiscount(customer, subtotal)
    couponDiscount := o.applyCouponDiscount(coupon, customerDiscount)
    return o.applyTax(couponDiscount)
}
```

5.2 Side-Effect-Free Functions

Definition: Side-Effect-Free Function

Functions that return a result without modifying any state. They can be called multiple times with the same result and can be safely composed together.

Go Implementation: Side-Effect-Free Functions

```
package domain

// Side-effect-free: only returns a value, no state change
func (p *Product) CalculatePrice(quantity int) (Money, error) {
    if quantity <= 0 {
        return Money{}, errors.New("quantity must be positive")
    }
    return p.unitPrice.Multiply(float64(quantity))
}

// Side-effect-free: query method
func (o *Order) CanBeCancelled() bool {
    return o.status == OrderStatusPending ||
           o.status == OrderStatusConfirmed
}

// Side-effect-free: returns new value instead of modifying
func (m Money) WithTaxRate(rate float64) (Money, error) {
    tax := m.Multiply(rate)
    return m.Add(tax)
}
```

5.3 Assertions

Definition: Assertion

> Explicit statements about the state of objects before and after operations. Assertions document what is guaranteed and help catch errors early.

Go Implementation: Assertions

```
package domain

import "fmt"

// Order with assertions
type Order struct {
    orderID  OrderID
    items    []OrderItem
    status   OrderStatus
}

// Invariant: Order always has at least one item when confirmed
func (o *Order) Confirm() error {
    // Pre-condition assertion
    if o.status != OrderStatusPending {
          return fmt.Errorf("pre-condition failed: order must be pending, got
%s", o.status)
    }

    if len(o.items) == 0 {
        return fmt.Errorf("pre-condition failed: order must have items")
    }

    // State change
    o.status = OrderStatusConfirmed

    // Post-condition assertion
    if o.status != OrderStatusConfirmed {
        return fmt.Errorf("post-condition failed: status should be confirmed")
    }

    // Invariant assertion
    if len(o.items) == 0 {
            return fmt.Errorf("invariant violated: confirmed order must have
items")
    }

    return nil
}
```

6. Practical Example: E-Commerce System

Let's put it all together with a complete e-commerce example showing how all the patterns work together.

Complete E-Commerce Domain Model

```go
package ecommerce

import (
    "context"
    "errors"
    "time"
)

// ============ Value Objects ============

type Money struct {
    amount   int64
    currency string
}

func (m Money) Add(other Money) (Money, error) {
    if m.currency != other.currency {
        return Money{}, errors.New("currency mismatch")
    }
    return Money{m.amount + other.amount, m.currency}, nil
}

func (m Money) Multiply(factor float64) Money {
    return Money{int64(float64(m.amount) * factor), m.currency}
}

// ============ Entities & Aggregates ============

type ProductID string
type OrderID string
type CustomerID string

type Product struct {
    id       ProductID
    name     string
    price    Money
    stock    int
}

func (p *Product) ID() ProductID { return p.id }
func (p *Product) Price() Money  { return p.price }

func (p *Product) Reserve(quantity int) error {
    if quantity > p.stock {
        return errors.New("insufficient stock")
    }
    p.stock -= quantity
    return nil
}

// Order is an Aggregate Root
type Order struct {
```

```
    items     []OrderItem
    status    OrderStatus
    total     Money
    createdAt time.Time
    events    []DomainEvent
}

type OrderItem struct {
    productID ProductID
    quantity  int
    unitPrice Money
}

type OrderStatus string

const (
    OrderStatusPending    OrderStatus = "PENDING"
    OrderStatusConfirmed  OrderStatus = "CONFIRMED"
    OrderStatusShipped    OrderStatus = "SHIPPED"
    OrderStatusDelivered  OrderStatus = "DELIVERED"
    OrderStatusCancelled  OrderStatus = "CANCELLED"
)

func NewOrder(id OrderID, customerID CustomerID) *Order {
    return &Order{
        id:         id,
        customerID: customerID,
        items:      make([]OrderItem, 0),
        status:     OrderStatusPending,
        createdAt:  time.Now(),
        events:     make([]DomainEvent, 0),
    }
}

func (o *Order) AddItem(product *Product, quantity int) error {
    if o.status != OrderStatusPending {
        return errors.New("cannot modify non-pending order")
    }

    if err := product.Reserve(quantity); err != nil {
        return err
    }

    o.items = append(o.items, OrderItem{
        productID: product.ID(),
        quantity:  quantity,
        unitPrice: product.Price(),
    })

    o.recalculateTotal()
    return nil
}

func (o *Order) Confirm() error {
```

```
    }
    if o.status != OrderStatusPending {
        return errors.New("order already confirmed")
    }

    o.status = OrderStatusConfirmed
    o.events = append(o.events, &OrderConfirmed{
        OrderID:    o.id,
        CustomerID: o.customerID,
        Total:      o.total,
        OccurredAt: time.Now(),
    })
    return nil
}

func (o *Order) recalculateTotal() {
    var total Money
    for _, item := range o.items {
        itemTotal := item.unitPrice.Multiply(float64(item.quantity))
        total, _ = total.Add(itemTotal)
    }
    o.total = total
}

func (o *Order) PullEvents() []DomainEvent {
    events := o.events
    o.events = nil
    return events
}

// ============ Domain Events ============

type DomainEvent interface {
    EventType() string
    OccurredAt() time.Time
}

type OrderConfirmed struct {
    OrderID    OrderID
    CustomerID CustomerID
    Total      Money
    OccurredAt time.Time
}

func (e *OrderConfirmed) EventType() string    { return "OrderConfirmed" }
func (e *OrderConfirmed) OccurredAt() time.Time { return e.OccurredAt }

// ============ Repository Interfaces ============

type OrderRepository interface {
    Save(ctx context.Context, order *Order) error
    FindByID(ctx context.Context, id OrderID) (*Order, error)
       FindByCustomer(ctx  context.Context,  customerID  CustomerID)  ([]*Order,
error)
```

```go
type ProductRepository interface {
    FindByID(ctx context.Context, id ProductID) (*Product, error)
    Save(ctx context.Context, product *Product) error
}

// ============ Domain Service ============

type OrderService struct {
    orders   OrderRepository
    products ProductRepository
    publisher EventPublisher
}

func (s *OrderService) PlaceOrder(
    ctx context.Context,
    customerID CustomerID,
    items []OrderItemRequest,
) (*Order, error) {
    // Create order
    order := NewOrder(generateOrderID(), customerID)

    // Add items
    for _, item := range items {
        product, err := s.products.FindByID(ctx, item.ProductID)
        if err != nil {
            return nil, err
        }
        if err := order.AddItem(product, item.Quantity); err != nil {
            return nil, err
        }
    }

    // Confirm order
    if err := order.Confirm(); err != nil {
        return nil, err
    }

    // Save
    if err := s.orders.Save(ctx, order); err != nil {
        return nil, err
    }

    // Publish events
    for _, event := range order.PullEvents() {
        s.publisher.Publish(ctx, event)
    }

    return order, nil
}

type OrderItemRequest struct {
    ProductID ProductID
```

7. Summary and Best Practices

Key Takeaways

Essential DDD Principles

- **Start with the domain:** Understand the business before writing code
- **Use Ubiquitous Language:** Speak the same language as domain experts
- **Model in code:** The code should reflect the domain model
- **Protect invariants:** Use aggregates to maintain consistency
- **Separate concerns:** Use layers to isolate domain logic
- **Embrace change:** The model should evolve with understanding

Best Practices Checklist

Table 3: DDD Best Practices

Area	Practice	Why
Language	Use domain terms in code	Reduces translation overhead
Entities	Keep identity simple	Easier to reason about equality
Value Objects	Make them immutable	Prevents unexpected changes
Aggregates	Keep them small	Easier to maintain consistency
Repositories	Return fully loaded aggregates	Avoids partial state issues
Services	Keep them stateless	Easier to test and reuse
Events	Make them immutable	Events represent facts

Testing	Test domain logic in isolation	Faster, more focused tests

Common Pitfalls to Avoid

Anti-patterns to Watch For

- **Anemic Domain Model:** Entities with only getters/setters and no behavior
- **God Objects:** Aggregates that try to do too much
- **Premature Abstraction:** Over-engineering before understanding the domain
- **Leaky Repositories:** Repositories that expose query details
- **Transaction Script:** Putting all logic in services instead of entities
- **Big Ball of Mud:** Not defining bounded contexts

Learning Path

1. **Start Small:** Apply DDD patterns to a single bounded context first
2. **Focus on Language:** Practice using Ubiquitous Language with your team
3. **Model Together:** Involve domain experts in modeling sessions
4. **Iterate:** Refactor the model as understanding grows
5. **Read:** Study the original DDD book and community resources
6. **Practice:** Apply patterns in real projects, learning from mistakes

Final Advice

Domain-Driven Design is not a one-size-fits-all solution. It's a set of principles and patterns that should be applied pragmatically. The goal is to create software that aligns with the business domain, not to follow patterns dogmatically. Start with the core domain, apply patterns where they add value, and evolve your design as you learn more about the domain.